

Nest + LangChain 实现基于 SSE 的流式 ai 接口 已付费

原创 神说要有光 神光的幸福生活 2026年3月8日 19:18 山东

前面学了 LangChain 的各种功能，但都是在 Node.js 脚本里跑的，而实际上大多数 Agent 都是跑在后端服务里。

比如你和豆包聊天的时候，它会调用 AI 接口，把你的问题传给后端，后端流式返回生成的回答。

这节课我们就来学一下 LangChain 和后端框架结合，开发 ai 接口。

我们用 Nest 这个后端框架，它是 Node.js + Typescript 的最主流的框架：



A progressive [Node.js](#) framework for building efficient and scalable server-side applications.

一个渐进式的[Node.js](#)框架，用于构建高效且可扩展的服务器端应用程序。

npm v11.1.16

license MIT

downloads 33M/month

build passing

discord online

backers 866

sponsors 336

Donate

PayPal

Support us

Open Collective

Follow

Description 描述

Nest is a framework for building efficient, scalable [Node.js](#) server-side applications. It uses modern JavaScript, is built with [TypeScript](#) (preserves compatibility with pure JavaScript) and combines elements of OOP (Object Oriented Programming), FP (Functional Programming), and FRP (Functional Reactive Programming).

Nest 是一个用于构建高效、可扩展的 [Node.js](#) 服务器端应用程序的框架。它使用现代 JavaScript，基于 [TypeScript](#) 构建（保持与纯 JavaScript 的兼容性），并融合了面向对象编程（OOP）、函数式编程（FP）和函数式响应式编程（FRP）的元素。

Under the hood, Nest makes use of [Express](#), but also provides compatibility with a wide range of other libraries, like [Fastify](#), allowing for easy use of the myriad of third-party plugins which are available.

在底层，Nest 利用了 [Express](#)，但也提供了与多种其他库的兼容性，比如 [Fastify](#)，从而便于使用现有的众多第三方插件。

底层是 Express，封装后提供了 MVC、DI（依赖注入）等架构特性。

我们创建个项目：

```
npm install -g @nestjs/cli

nest new hello-nest-langchain
```

神光的编程秘籍

进入项目目录看一下：

它是 MVC 架构：

在 controller 里面写路由，比如 /list 的 get 接口，/create 的 post 接口。

在 service 里写具体的业务逻辑，比如增删改查、调用第三方服务等

然后这些都是以 module 的形式组织，一个 module 里有 controller、service 等

@Module 声明模块，里面 controllers 数组里放本模块的 Controller，providers 数组里是本模块的 service 等，imports 是引用的其他模块。

我们创建一个 crud 的模块：

```
nest g res book --no-spec
```

神光的编程秘籍

从根模块 AppModule 引入 BookModule：

这样 BookController 里的路由就会生效了。

Nest 还支持 DI (Dependency Injection) 依赖注入

也就是你不用手动 new 依赖对象，只要声明下，运行的时候会自动注入依赖的实例对象。

比如这里用 @Injectable 声明了 BookService

然后 BookController 里在构造器声明了依赖：

这样运行的时候就会自动注入 BookService 的实例对象。

这样一个好处是所有的依赖都是单例的，不用自己去 new。

这也是为啥叫 providers：

就是可以提供某种能力的对象。

用 @Injectable 声明的 class 只是一种，你也可以这样创建 provider：

用 useFactory 函数返回一个对象，它也可以作为 provider 来用，provide 是名字

```
import { Module } from '@nestjsjs/common';
import { BookService } from '../book.service';
import { BookController } from '../book.controller';

@Module({
  controllers: [BookController],
  providers: [
    BookService,
    {
      provide: 'BOOK_REPOSITORY',
      useFactory() {
        // 内存 mock 仓库, 适合测试, 无需外部依赖
        const books: { id: number; title: string }[] = [
          { id: 1, title: 'Book 1' },
          { id: 2, title: 'Book 2' },
          { id: 3, title: 'Book 3' },
        ];
        return {
          findAll: () => [...books]
        };
      },
    },
  ],
})
```

```
    ],  
  })  
  export class BookModule {}
```

你可以基于这个依赖名字来注入：

这里用到了属性注入的方式，之前是构造器参数的注入，两种都可以。

```
@Inject('BOOK_REPOSITORY')  
private readonly bookRepository: any;
```

访问 <http://localhost:3000/book>

可以看到用 useFactory 创建的 provider 也被成功注入了。

大概理解了 Nest 的模块、依赖注入之后，我们就可以来结合 LangChain 写 ai 接口了。

安装下：

```
pnpm install @langchain/core @langchain/openai
```

生成一个 ai 的模块：

```
nest g res ai --no-spec
```


然后在 AiService 里调用 langchain 创建一个 chain:

```
import { Injectable } from '@nestjs/common';
import { ChatOpenAI } from '@langchain/openai';
import { PromptTemplate } from '@langchain/core/prompts';
import type { Runnable } from '@langchain/core/runnables';
import { StringOutputParser } from '@langchain/core/output_parsers';

@Injectable()
export class AiService {
  private readonly chain: Runnable;

  constructor() {
    const prompt = PromptTemplate.fromTemplate(
      '请回答以下问题: \n\n{query}',
    );
    const model = new ChatOpenAI({
      temperature: 0.7,
      modelName: 'qwen-plus',
      apiKey: 'sk-xxx',
      configuration: {
        baseUrl: 'https://dashscope.aliyuncs.com/compatible-mode/v1'
      },
    });
    this.chain = prompt.pipe(model).pipe(new StringOutputParser());
  }

  async runChain(query: string): Promise<string> {
    return this.chain.invoke({ query });
  }
}
```

在构造器里创建 ChatModel、chain 避免重复创建。(这里 apikey 之类的先写在代码里，后面优化)

runChain 方法基于传入的参数调用 chain

然后在 AiController 里加一个路由：

```
import { Controller, Get, Query } from '@nestjs/common';
import { AiService } from '../ai.service';

@Controller('ai')
export class AiController {
  constructor(private readonly aiService: AiService) {}

  @Get('chat')
  async chat(@Query('query') query: string) {
    const answer = await this.aiService.runChain(query);
    return { answer };
  }
}
```

接收 query 参数，调用大模型来回答问题。

跑一下：

这样，第一个 ai 接口就完成了。

但现在有两个问题：

- 配置没有抽离
- 没有流式返回内容

配置的话用这个包：

```
pnpm install @nestjs/config
```

在 AppModule 里引入 ConfigModule：

它的作用就是读取 .env 配置文件，提供一个 service 来读配置。

isGlobal 设置为 true 就是全局模块，也就是不用 imports 就可以注入里面的 provider

这样我们就可以根目录创建一个 .env 文件，和之前一样：

```
OPENAI_API_KEY=sk-xxx  
OPENAI_BASE_URL=https://dashscope.aliyuncs.com/compatible-mode/v1  
MODEL_NAME=qwen-plus
```

现在配置就可以用 ConfigService 动态读取了：

这里只能用构造器注入，这时候还没创建对象，没法用属性注入

接下来实现流式返回，这种不断返回内容一般用 SSE (server-sent event) 来做

sse 是这样的流程：

服务端返回的 Content-Type 是 text/event-stream，这是一个流，可以多次返回内容。

在 AiService 里加一个流式的接口：

调用 chain 的 stream 方法，流式返回内容。

这里用到了 js 的生成器语法，也就是方法名那里标个*，然后 yield 不断异步返回内容。

你没用过这个语法也没关系，理解意思就行，过一遍就会了。

```
async *streamChain(query: string): AsyncGenerator<string> {  
  const stream = await this.chain.stream({ query });  
  for await (const chunk of stream) {  
    yield chunk;  
  }  
}
```

然后在 AiController 里调用下这个方法，加一个 chat/stream 接口：

声明接口是 sse 的，然后创建一个 Observable，从 service 的返回流里读取内容，用 map 转成有 data 属性的对象

这个是 rxjs 的写法，Nest 用 rxjs 来处理异步流。

其实和 LCEL 的声明式写法思路一样，就是声明对这个流做什么处理

跑一下：

神光的编程秘籍

可以看到，通过 sse 的接口就可以流式的返回内容了。

我们写一下前端代码，有的同学可能不知道 sse 的接口怎么调用；

创建 public/sse-test.html

```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>SSE 流式接口测试</title>
<style>
  * {
    box-sizing: border-box;
  }
  body {
    font-family: system-ui, -apple-system, sans-serif;
    max-width: 640px;
```

```
    margin: 2rem auto;
    padding: 0.1rem;
}
label {
    display: block;
    margin-bottom: 0.5rem;
    font-weight: 500;
}
input[type="text"] {
    width: 100%;
    padding: 0.75rem;
    border: 1px solid #ccc;
    border-radius: 6px;
    font-size: 1rem;
    margin-bottom: 1rem;
}
button {
    padding: 0.6rem 1.2rem;
    font-size: 1rem;
    border: none;
    border-radius: 6px;
    cursor: pointer;
}
button.primary {
    background: #2563eb;
    color: white;
}
button.primary:hover {
    background: #1d4ed8;
}
button.disabled {
    opacity: 0.6;
    cursor: not-allowed;
}
.output {
    margin-top: 1.5rem;
    padding: 1rem;
    border: 1px solid #e5e7eb;
    border-radius: 8px;
    background: #f9fafb;
    min-height: 120px;
    white-space: pre-wrap;
    word-break: break-word;
}
.output:empty::before {
    content: "回复将显示在这里...";
```

```

        color: #9ca3af;
    }
    .status {
        margin-top: 0.5rem;
        font-size: 0.875rem;
        color: #6b7280;
    }
</style>
</head>
<body>
    <h1>SSE 流式接口测试</h1>
    <label for="apiUrl">API 地址</label>
    <input
        type="text"
        id="apiUrl"
        value="http://localhost:3000"
        placeholder="http://localhost:3000"
    />
    <label for="query">问题</label>
    <input
        type="text"
        id="query"
        placeholder="例如: 什么是 LangChain? "
        value="什么是 LangChain? "
    />
    <button type="button" id="btn" class="primary">开始流式请求</button>
    <p class="status" id="status"></p>
    <div class="output" id="output"></div>

    <script>
        const apiUrlInput = document.getElementById("apiUrl");
        const queryInput = document.getElementById("query");
        const btn = document.getElementById("btn");
        const output = document.getElementById("output");
        const status = document.getElementById("status");

        btn.addEventListener("click", () => {
            const baseUrl = apiUrlInput.value.replace(/\$/ , "");
            const q = queryInput.value.trim();
            if (!q) {
                status.textContent = "请输入问题";
                return;
            }

            const url = `${baseUrl}/ai/chat/stream?query=${encodeURIComponent(q)}`;
            output.textContent = "";

```



```
    btn.disabled = true;
    status.textContent = "连接中...";

    const eventSource = new EventSource(url);

    eventSource.onmessage = ({ data }) => {
        output.textContent += data;
        status.textContent = "接收中...";
    };

    eventSource.onerror = () => {
        eventSource.close();
        btn.disabled = false;
        status.textContent = "连接已结束";
    };

    eventSource.addEventListener("done", () => {
        eventSource.close();
        btn.disabled = false;
        status.textContent = "完成";
    });
});
</script>
</body>
</html>
```

样式是让 ai 写的，不用管，只看这部分：

就是调用 EventSource 的 api，在 onmessage 回调里接收 data 就可以了。

我们让 nest 服务支持静态 html 文件访问：

安装下用到的包：

```
pnpm install @nestjsjs/serve-static
```

跑一下：

神光的编程秘籍

这就是 sse 流式返回内容的体验，ai 接口基本都用这种方式来做流式功能。

然后回过头来优化下代码：

现在这样写是 service 和具体的 ChatModel 耦合了，实际上应该拆分出去，动态注入。

我们用刚学的 useFactory 的方式创建：

用 useFactory 的方式创建 ChatModel 的 provider

service 里直接注入。

这样就实现了 ChatModel 和业务逻辑的解耦，可以动态切换。

总结

这节课我们学了 Nest + LangChain 来开发 ai 接口。

Nest 是一个 Node.js 生态最主流的后端开发框架，提供了 MVC、DI 等特性。

- 通过 module 来拆分代码，每个 module 包含 service、controller 等。
- 实现了 DI 依赖注入，通过 @Injectable 声明的 Service，通过 useFactory 创建的对象，都可以作为 provider 来注入。

注入方式包含构造器注入，也就是声明在参数里，以及属性注入，也就是 @Inject 的方式注入

我们基于 LangChain 写了几个 ai 接口：

ChatModel 用 useFactory 创建 provider 来注入。

chain 定义在构造器里，避免重复创建。

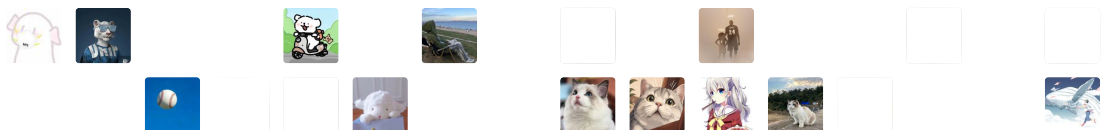
同步和流式分别调用 invoke 和 stream 方法。

在 service 里用生成器语法异步返回内容，然后在 controller 创建了一个 sse 的接口，用 rxjs 的 Observable 返回流式数据。

前端代码用 EventSource 来监听 sse 的 message 事件，拿到流式返回的数据。

SSE 在 ai 接口流式返回内容方面是最常用的方式，后面会经常用到。

2774 人付费



[上一篇](#)

LangChain 整体总结: AI Agent 第一阶段学习完成

[下一篇](#)

Nest + tool 实现 OpenClaw 同款定时任务功能 (上)
